

kvkk-rag

KVKK (Kişisel Verileri Koruma Kurumu) **kurul karar özetleri** üzerinde semantik arama ve analiz. Kararların tamamı indekslenir, sorgular hem anlam (vektör) hem terim (Türkçe full-text) tarafından aranır ve iki sonuç listesi RRF ile birleştirilir.

İki mod var:

Mod	Ne yapar	Maliyet
Arama	İlgili kurul kararlarını bulur, eşleşen pasajı vurgular, kararın hükmünü ve yaptırımını gösterir	Ücretsiz
Analiz	Aynı retrieval + LLM'in yalnızca bulunan kararlara dayanarak ürettiği, karar numarasıyla belgelenmiş cevap	Soru başına ~\$0,012

- **Stack:** Next.js 15 (App Router), TypeScript, Node 20+, Tailwind v4
- **Vektör deposu:** Supabase Postgres + **pgvector** (HNSW, cosine)
- **Embedding:** Hugging Face Inference API — [intfloat/multilingual-e5-large](#) (1024d, açık kaynak, ücretsiz)
- **Cevap üretimi:** OpenAI [gpt-4.1](#) (yalnızca Analiz modunda)
- **Tüm key'ler server-side.** Hiçbir değişken [NEXT_PUBLIC_](#) almaz; tarayıcı ne HF'e, ne OpenAI'a, ne Postgres'e doğrudan bağlanır — yalnızca [/api/search](#) ve [/api/chat](#) 'e gider. [OPENAI_API_KEY](#) yoksa arama çalışmaya devam eder, yalnızca Analiz sekmesi devre dışı kalır.

Klasör yapısı

```
drizzle/
  0000_init.sql          # şema: pgvector, unaccent'li Türkçe TS config, tablolar, GIN
scripts/
  00-verify-db.ts        # pgvector + HNSW + turkish full-text desteği kontrolü
  01-migrate.ts           # drizzle/*.sql dosyalarını DIRECT_URL üzerinden koşar
  02-scrape.ts            # kvkk.gov.tr -> data/decisions.json (resume'lu)
  03-embed.ts             # chunk + embed -> Postgres (resume'lu)
  04-build-indexes.ts     # HNSW indeksi (veri yüklendikten SONRA) + ANALYZE
  05-search-cli.ts        # terminalden arama (UI'sız kalite kontrolü)
  99-reset.ts             # kvkk şemasını düşür (model/boyut değişikliği için)
src/
  config.ts               # model/boyut/chunk/arama parametreleri – tek kaynak
  chunk.ts                # Türkçe cümle sınırına duyarlı chunking
  extract.ts              # hüküm bloğu + yaptırım çıkarımı (regex, ücretsiz)
  chat.ts                 # retrieval + OpenAI cevap üretimi (Analiz modu)
  corpus.ts               # UI için indeks durumu
  db/{schema,client,direct}.ts
  embeddings/hf.ts         # HF feature-extraction + E5 prefix'leri
  scrape/kvkk.ts           # liste + detay sayfası parser'ları
  search/hybrid.ts         # pgvector + full-text, RRF füzyonu
app/
  page.tsx                # ana sayfa (server component)
  components/
    Workspace.tsx          # Arama / Analiz sekmeleri
    SearchBar.tsx          #
    Chat.tsx               # akışlı cevap + tıklanabilir atıflar
  api/search/route.ts      #
  api/chat/route.ts        # NDJSON akışı: sources -> delta... -> done
data/                      # scrape çıktısı (gitignore)
```

Kurulum

1) Bağımlılıklar

```
npm install
```

2) Ortam deęişkenleri

```
cp .env.example .env
```

Deęişken	Açıklama
HF_TOKEN	Hugging Face token ("Read" yetkisi yeterli)
HF_EMBEDDING_MODEL	intfloat/multilingual-e5-large (default)
HF_EMBEDDING_DIM	1024 (e5-large) / 768 (e5-base)
OPENAI_API_KEY	Opsiyonel. Yalnızca Analiz modu için; yoksa arama çalışır
OPENAI_CHAT_MODEL	gpt-4.1 (default)
DATABASE_URL	Supabase pooler (port 6543) — uygulama sorguları
DIRECT_URL	Supabase direct (port 5432) — migration / DDL

⚠ **.env** yalnızca sunucu başlangıcında okunur. **OPENAI_API_KEY** ekledikten sonra `npm run dev`'i yeniden başlatmadan Analiz sekmesi görünmez.

⚠ **Pooler uyarısı:** 6543 transaction pooler'dır, prepared statement desteklemez. `src/db/client.ts` bağlantıyı `prepare: false` ile kurar. DDL (CREATE EXTENSION / CREATE INDEX) pooler üzerinden güvenilir çalışmaz; bu yüzden migration ve index script'leri **DIRECT_URL** kullanır. Bu ikisini karıştırırsan işlemler anlaşılabilir hatalarla patlar.

Çalıştırma sırası

```
npm run verify:db # pgvector, HNSW, 'turkish' full-text desteęi raporu
npm run db:migrate # kvkk şeması + tablolar + GIN index (DIRECT_URL)
npm run scrape # data/decisions.json (36 sayfa -> 310 karar)
npm run embed # chunk + HF embedding -> Postgres (3.757 pasaj, ~6 dk)
npm run db:index # HNSW vektör indeksi + ANALYZE
```

`npm run embed` varsayılan olarak 2 eşzamanlı HF isteęi kullanır; `-- --concurrency 4` ile hızlandırılabilir (ölçülen: ~6,6 pasaj/s).

Ardından:

```
npm run dev
# http://localhost:3000
```

Terminalden hızlı kalite kontrolü:

```
npm run search -- "güvenlik kamerasıyla ses kaydı alınması"
```

Kesintiye dayanıklılık

`scrape` ve `embed` yeniden koşulabilir ve kaldığı yerden devam eder:

- `scrape` yalnızca `data/decisions.json` içinde olmayan id'leri indirir (`--force` ile hepsini yeniden indirir), her 25 kayıta diske yazar.
- `embed` yalnızca `embedding IS NULL` olan chunk'ları işler. HF rate limit'e girip koşu yarıda kalırsa `npm run embed` komutunu tekrar çalıştır.

Sistemi nasıl kurduk — aşama aşama

Özet

0. Keşif	kaynak sitenin yapısını çöz	→ 36 sayfa, 313 link, düz HTML
1. Doğrulama	altyapı gerçekten destekliyor mu?	→ pgvector 0.8.2, HNSW, turkish FTS, HF token
2. Şema	tablolar + Türkçe FTS konfigürasyonu	→ kvkk.decisions, kvkk.chunks
3. Scrape	kararları indir	→ 310 karar, ort. 9.688 karakter
4. Chunk	aranabilir parçalara böl	→ 3.757 pasaj (ort. 12,1/karar)
5. Embed	her pasajı vektöre çevir	→ 1024 boyut, ~6 dk
6. İndeks	HNSW + GIN + stoplex	→ 29 MB + 5,4 MB
7. Retrieval	vektör + terim, RRF ile birleştir	→ ~0,9 s ısınmış
8. Sunum	alıntı, vurgulama, hüküm çıkarımı	→ 298/310 hüküm
9. Generation	bulunan kararlara dayalı cevap	→ ~\$0,012/soru

Aşağıda her aşamanın ne yaptığı, **neden** öyle yapıldığı ve ölçülen sonucu var.

0. Kaynağı keşfetmek

Kod yazmadan önce kaynak sitenin nasıl çalıştığını çözdük:

- Kararlar `?page=N` ile sayfanılıyor, pagination'daki `»` linki son sayfayı veriyor → **36 sayfa** (scraper bunu her koşuda yeniden okur, sabit değil).
- Liste sayfası markup'ı: `.members__item .members__item-meta` → `h2` (başlık) + `a.read-more` (detay linki).

- Detay sayfası: `.news__detail-article` → başlık + meta tablosu (Karar Tarihi / Karar No / Konu Özeti) + gövde paragrafları.
- Kritik bulgu:** sayfalar server-rendered. Düz `fetch` tam HTML döndürüyor, yani tarayıcı otomasyonu gerekmiyor → scrape dakikalar yerine saniyeler sürer.

1. Altyapıyı doğrulamak (`npm run verify:db`)

Şema yazmadan önce sunucunun gerekenleri gerçekten desteklediğini ölçtük. Varsaymak yerine ölçmek önemliydi: eksik bir eklenti migration'ı ya patlatır ya da sessizce yanlış konfigürasyon üretir.

Kontrol	Sonuç
Postgres	17.6
pgvector	0.8.2 (HNSW ve <code>iterative_scan</code> destekli)
<code>turkish</code> full-text konfigürasyonu + <code>turkish_stem</code>	✓
HF token / model boyutu	e5-large → 1024d ✓

2. Şemayı kurmak (`npm run db:migrate`)

`drizzle/*.sql` dosyaları `DIRECT_URL` (5432) üzerinden sırayla koşar. Üç önemli tercih:

- Ayrı `kvkk` şeması.** Bu Supabase örneği başka bir projeye paylaşımlı; `public` altına yazmak çakışma riski taşıyordu ve `db:reset` yanlış veriyi silebilirdi.
- Türkçe + diyakritik-duyarsız FTS konfigürasyonu.** `kvkk.turkish_unaccent` = `turkish` + `unaccent` sözlüğü. Kullanıcılar "guvenlik kamerasi" yazıyor, metinde "güvenlik kamerası" geçiyor. Ölçtük: ASCII yazım artık eşleşiyor, ekler stem'leniyor (`kararlarında` → `karar`), karar no tek token kalıyor (`2024/2196`).
- `tsvector` GENERATED kolonu.** `chunks.tsv` her INSERT'te otomatik dolar; ayrı bir güncelleme adımı ve tutarsızlık riski yok.

3. Kararları indirmek (`npm run scrape`)

Liste sayfaları gezilir, ardından her detay sayfası indirilip parse edilir.

- Veritabanına dokunmaz:** bu adımın tek çıktısı `data/decisions.json` — diskte bir JSON dosyası. Kararların DB'ye yazılması burada değil, bir sonraki adımda (`npm run embed`) olur.
- Kibar davranır:** istekler arası 400 ms, hatalarda exponential backoff.
- Kaldığı yerden devam eder:** `data/decisions.json` 'daki id'ler tekrar indirilmez, her 25 kayıttan sonra yazılır.
- Sonuç:** 313 linkten **310 karar** (2018–2026), ortalama 9.688 karakter. Kalan 3'ü kaynak sitede kırık link (anasayfaya yönlendiriliyor).

Bu aşamada iki veri kusuru bulup düzelttik: eski format sayfalarda `:` ayrı hücrede değil değerin başında geliyordu (55 karar `: 2020/86` olarak kaydedilmişti) ve bazı sayfalarda etiket değerin içinde tekrar ediyordu (`Konu Özeti : ...`).

4. Chunking (`src/chunk.ts`)

Kararlar ortalama ~10 bin karakter; tek vektörle temsil edilirse spesifik bir gerekçe kaybolur. Bu yüzden **1200 karakter hedef + 200 karakter overlap** ile parçalanır.

Neden 1200: e5 modelleri 512 token'da kesiyor, Türkçe'de multilingual tokenizer ~3 karakter/token üretiyor → ~1500 karakter güvenli üst sınır.

Neden cümle sınırında: kararlar "...12 nci maddesinin (1) numaralı fıkrasında yer alan yükümlülüğü" gibi tek cümlede taşınan hukuki gerekçeler içeriyor. Sabit karakter penceresi bunları ortadan keserse chunk anlamsızlaşır. Kısaltma tuzakları (`md.` , `No.` , `12.`) elle geçilir.

Sonuç: 3.757 pasaj, ortalama 12,1 pasaj/karar.

5. Veritabanına yazma + embedding (`npm run embed`)

Kafa karıştıran nokta burada netleşiyor. Bir önceki adım (`scrape`) veritabanına hiç dokunmadı; elimizde yalnızca `data/decisions.json` var. Kararların ve chunk'ların DB'ye yazılmasının **tamamı** bu tek script'te (`scripts/03-embed.ts`) ve sırayla üç alt-adımda olur:

1. **Kararlar** → `kvkk.decisions` . Her karar *ham ve bütün* haliyle yazılır: tam gövde (`body`), başlık, karar no/tarihi, meta ve regex ile çıkarılan hüküm/yaptırım. **Bu tabloda vektör yoktur** — burası gösterim ve alıntı kayıdır, arama birimi değil.
2. **Chunk'lar** → `kvkk.chunks` . `chunkText(body)` her kararı pasajlara böler ve pasajlar bu tabloya yazılır — **ama embedding kolonu bu noktada boş (NULL)** . Yani chunk önce yalnızca *metin* olarak kaydedilir.
3. **Embedding**. Ayrı bir geçişte yalnızca `embedding IS NULL` olan pasajlar HF'e gönderilir; dönen 1024 boyutlu vektör `UPDATE` ile aynı satıra yazılır.

Yani chunk'lar "tek hamlede vektörlenmiş halde" değil, **önce metin, sonra vektör** olarak iki geçişte yazılır. Bunu bilerek ayırdık: chunk metnini yazmak ucuz (yerel DB işlemi), vektör üretmek pahalı (uzak API + rate limit). Ayrı oldukları için koşu yarıda kalsa bile `npm run embed` 'i tekrar çalıştırmak yalnızca eksik kalan vektörleri tamamlar, baştan başlamaz.

head + text — embedding'e tam olarak ne gönderiyoruz? Her chunk satırı iki ayrı metin alanı taşır:

- **text** = pasajın kendisi; kararın gövdesinden kesilen ~1200 karakterlik parça. Kullanıcının UI'da okuduğu, vurgulanan snippet **budur**.
- **head** = kararın **Konu Özeti** (yoksa başlığı), her chunk'a kopyalanan (denormalize edilen) bağlam başlığı. Kararın "neyle ilgili" olduğunu söyler.

HF'e giden metin ikisinin birleşimidir: `head + "\n\n" + text` . Neden ikisi birden: karar gövdesi "veri sorumlusu", "ilgili kişi" gibi jenerik hukuki ifadelerle yazılmış; olayın hangi sektörde geçtiği (hastane, banka, kargo...) çoğu zaman **yalnızca Konu Özeti'nde** geçiyor. `head` olmadan "hastane kayıtları" sorgusu, gövdesinde "hastane" kelimesi hiç geçmeyen doğru kararı bulamaz. Aynı `head` full-text `tsv` 'ye de girer (böylece terim araması da başlıktan yakalar). Ama **kullanıcıya gösterilen pasaj yalnızca text** — başlığı `text` 'in içine gömseydik her snippet aynı cümleyle başlardı.

- **E5 prefix'leri zorunlu:** doküman `passage:` , sorgu `query:` . Bu modeller asimetric eğitildi; prefix'i atlamak recall'u belirgin düşürür.
- **Kaldığı yerden devam eder:** yalnızca `embedding IS NULL` olan pasajlar işlenir. Metin değişirse embedding otomatik `NULL` 'a çekilir ve yeniden üretilir.

- **Batch'li yazma:** DB uzak (Mumbai). Satır başına bir INSERT ~2.500 round-trip demektir; çok satırlı INSERT ile tek istekte 200 satır yazılıyor.

Sonuç: 3.757 pasajın tamamı embed edildi, 0 başarısız batch. Ölçülen hız ~6,6 pasaj/s (4 eşzamanlı istek) — tek seferde 3.661 pasaj 385 saniyede bitti.

6. İndeksleme (`npm run db:index`)

Veri yüklendikten **sonra** kurulur — boş tabloya index kurup 3.757 satır yazmak hem yavaş hem daha düşük kaliteli bir HNSW grafi üretir.

İndeks	Boyut	İşi
<code>chunks_embedding_hnsw_idx</code>	29 MB	vektör benzerliği (<code>vector_cosine_ops</code>)
<code>chunks_tsv_idx</code> (GIN)	5,4 MB	Türkçe full-text
<code>kvkk.stoplex</code>	46 satır	vurgulamada elenecek jenerik lexeme'ler

`stoplex` bu aşamada `ts_stat` ile hesaplanır (DF > %25). Arama sırasında hesaplamak pahalı olurdu; korpus başına bir kez yeter.

7. Retrieval (`src/search/hybrid.ts`)

Tek SQL statement'ında iki kol koşar, sonuç TypeScript'te birleştirilir:

1. **Vektör kolu:** HNSW cosine, `LIMIT 60`
2. **Terim kolu:** GIN + `ts_rank_cd` , `LIMIT 60`
3. **RRF füzyonu:** `score += 1/(60 + sıra)` — skorlar farklı ölçeklerde olduğu için sadece **sıra** kullanılır, kalibrasyon gerekmez
4. **MaxP:** kararın skoru = en iyi pasajının skoru (toplamak uzun kararları sistematik olarak öne çıkarırdı)

Latency'yi burada 3,7 s'den ~0,9 s'ye indirdik: sorgular zaten hızlıydı (HNSW 30 ms, GIN 1 ms), darboğaz round-trip sayıydı.

8. Sunum katmanı

Retrieval doğru sonucu bulduktan sonra, sonucun **okunabilir ve alıntılanabilir** olması gerekiyor:

- **Vurgulama** eşleşme sorgusundan ayrı bir (gevşek, OR) sorguyla yapılır — aksi halde vektörle bulunan sonuçlarda hiç vurgu çıkmıyor ve pasaj cümlelerin ortasından başlıyordu.
- **Jenerik terimler elenir** (`stoplex`), yoksa `veri` / `kişisel` her yerde işaretleniyordu.
- **Hüküm çıkarımı** (`src/extract.ts`): kararın operatif bloğu ve yaptırım etiketi regex ile çıkarılır — LLM çağırısı yok, dolayısıyla maliyeti yok. Kapsam: 298/310 hüküm, 247/310 yaptırım, 101/310 ceza tutarı.

9. Cevap üretimi (`src/chat.ts`) — Analiz modu

Bu son aşama ve tek ücretli bileşen. Akış:

kullanıcı sorusu

- hybridSearch (aynı ücretsiz retrieval, 6 karar)
- bağlam: karar no + tarih + konu + yaptırım + pasaj + hüküm
- gpt-4.1 (temperature 0.2, "yalnızca verilen kararları kullan")
- NDJSON akışı: sources → delta... → done(usage)

Halüsinasyona karşı üç katman:

1. Sistem prompt'u modele yalnızca verilen kararları kullanmasını, yetmiyorsa bunu açıkça söylemesini emreder.
2. Her iddia karar numarasıyla belgelenir ([2024/2196]).
3. **UI atıfları doğrular:** kaynak listesinde olmayan bir karar numarası link yapılmaz, sarı uyarı olarak işaretlenir. Testlerde uydurma atıf sayısı 0.

Ölçülen: ~3.200 girdi + ~700 çıktı token ≈ **\$0,012/soru**. UI her cevabın altında gerçek token kullanımını gösterir.

Tasarım kararları

Neden hybrid (vektör + full-text)? Hukuki aramada iki farklı ihtiyaç var: "biyometrik veriyle mesai takibi" gibi kavramsal sorgular vektör tarafını, "2024/2196", "12 nci madde", "fidye yazılımı" gibi tam terimler full-text tarafını gerektiriyor. Tek başına vektör araması ikinci grubu kaçırıyor.

Neden RRF (Reciprocal Rank Fusion)? İki kolun skorları farklı ölçeklerde (cosine 0–1, `ts_rank_cd` sınırsız). Skorları normalize edip toplamak ölçek varsayımlarına bağımlı kalır; RRF yalnızca **sırayı** kullanır, kalibrasyon gerektirmez.

Neden `turkish_unaccent` text search config? Kullanıcılar "guvenlik", "sifreleme" gibi ASCII yazımlar deniyor. `unaccent` + `turkish_stem` mapping'i hem diyakritikleri hem ekleri normalize ediyor ("kararlarında" → "karar"). `to_tsvector(regconfig, text)` IMMUTABLE olduğu için bu config GENERATED kolonda kullanılabilir.

Neden chunk seviyesinde arama, karar seviyesinde sonuç? Kararlar ortalama ~12 bin karakter; tek vektörle temsil edilirse spesifik bir gerekçe kayboluyor. Chunk'lar aranıp **MaxP** (kararın skoru = en iyi chunk'ının skoru) ile karar seviyesine çıkılıyor. Skor toplamak uzun kararları sistematik olarak öne çıkarırdı.

Neden `chunks.head` ayrı bir kolon? Karar gövdesi "veri sorumlusu", "ilgili kişi" gibi genel ifadelerle yazılmış; hangi sektör/olay olduğu çoğu zaman yalnızca başlıkta geçiyor. Bu yüzden kararın Konu Özeti her chunk'a denormalize edilir ve **hem embedding girdisine hem tsvector'e** girer — "hastane kayıtları" gibi bir sorgu böylece doğru kararı bulur. Ama UI'da gösterilen pasaj yalnızca `text`; başlığı chunk metnine gömmek snippet'in başlığı tekrar etmesine yol açardı. (GENERATED kolon başka tabloya bakamadığı için denormalizasyon zorunlu.)

Neden E5 prefix'leri? `intfloat/multilingual-e5-*` modelleri asimetrik eğitildi: doküman `passage:`, sorgu `query:` prefix'i ister. Prefix'i atlamak recall'u belirgin düşürür. `src/config.ts` model adında "e5" yoksa prefix'i otomatik kapatır.

Neden HNSW migration'da değil? Boş tabloya index kurup 3–4 bin satır yazmak, veriyi yazıp sonra index kurmaktan hem yavaş hem daha düşük kaliteli graf üretir. Bu yüzden `npm run db:index` ayrı adım.

Neden iki arama kolu tek SQL statement'ında? Sorguların kendisi hızlı (ölçülen: HNSW ~30 ms, GIN ~1 ms) ama DB uzak — boş bir `SELECT 1` bile ~250 ms. Kolları ayrı ayrı, üstelik `SET LOCAL` için transaction içinde koşturmak 5–6 round-trip demekti (~3,7 s). Tek statement + postgres-js'in transaction pipeline'ı bunu ~0,9 s'ye indirdi.

⚠ **HNSW_EF_SEARCH ile CANDIDATE_POOL bağlantılı.** HNSW bir sorguda `ef_search` 'ten fazla satır döndürmez. pgvector varsayılanı 40; pool 60 iken `SET LOCAL hnsw.ef_search` atlanırsa LIMIT'e rağmen sessizce 40 aday gelir ve füzyon zayıflar. `src/config.ts` bu yüzden ikisini birlikte tutuyor.

Neden vurgulama, eşleşme sorgusundan ayrı bir sorgu kullanıyor? Eşleşmede `websearch_to_tsquery` terimleri AND'liyor — seçici olması istediğimiz davranış. Ama aynı sorguyu gösterimde kullanmak, vektörle bulunan sonuçlarda `tsv @@ q` false olduğu için `ts_headline` 'ı tamamen devre dışı bırakıyordu: pasaj chunk'ın ham başından alınıyor ve overlap yüzünden cümlelerin ortasından başlıyordu ("yandan, Kanun'un..."). Vurgulama artık gevşek (OR) bir sorguyla yapılıyor.

Neden kvkk.stoplex ? Gevşek sorgu "veri", "kişisel" gibi terimleri de işaretliyordu — ölçüm (`ts_stat` , 3.757 chunk): `ver` %92,5, `kisisel` %86,6, `ilgil` %84,5. Korpusun %92'sinde geçen bir kelimeyi vurgulamak "bu sonuç neden ilgili" sorusuna cevap vermiyor, sadece pasajı okunmaz yapıyor. `npm run db:index` DF > %25 olan lexeme'leri (46 tane) bu tabloya yazıyor ve vurgulama sorgusundan eleniyor. Sorgunun tamamı jenerik terimden oluşuyorsa (ör. "kişisel veri") hepsine geri düşülüyor — hiç vurgu olmamasından iyi.

Neden hüküm ayrı bir blok olarak çıkarılıyor? KVKK kararları tutarlı bir kuyruk yapısı izliyor: gerekçe maddeleri (`<ul>`) → "hususları dikkate alındığında..." operatif paragraf → `karar verilmiştir`. Bu son cümle çoğu zaman 18 karakterlik ayrı bir paragraf (310 kararın 203'ünde), dolayısıyla yalnızca onu almak hiçbir bilgi vermiyor. `src/extract.ts` bloğun sonunu "karar veril..." ile, başını operatif açılış kalıbıyla bulur; kalıp yoksa karakter bütçesiyle geriye gider. Kapsam: **298/310**.

Neden yaptırım yalnızca hüküm bloğundan çıkarılıyor? "idari para cezası", "şikâyetin reddine" gibi ifadeler kararın anlatı kısmında da geçiyor (tarafların iddiaları, mevzuat alıntıları, önceki kararlara atıflar). Gövdenin tamamına bakmak yanlış etiket üretiyordu.

Model / boyut değiştirmek

`HF_EMBEDDING_DIM` şemadaki `vector(N)` kolonunu belirler, yani boyut değişikliği şema değişikliğidir:

```
# .env içinde HF_EMBEDDING_MODEL + HF_EMBEDDING_DIM güncelle, sonra:
npm run db:reset -- --yes # kvkk şemasını düşürür; data/decisions.json'a dokunmaz
npm run db:migrate
npm run embed # scrape'i tekrarlamaya gerek yok
npm run db:index
```

`npm run verify:db` tablodaki boyutla `.env` 'in uyuşmadığını tespit edip uyarır.

Maliyet ve sınırlar

Ölçülen durum (310 karar / 3.757 pasaj / 1024 boyut):

Bileşen	Servis	Durum
Embedding	HF Inference API	Ücretsiz katman — indeksleme 3.757 çağrı (tek seferlik), arama başına 1 çağrı
Veritabanı	Supabase Postgres + pgvector	<code>kvkk</code> şeması 69 MB (500 MB ücretsiz limitten)
Hüküm / yaptırım özeti	—	Deterministik regex çıkarımı (<code>src/extract.ts</code>), API çağrısı yok
Cevap üretimi (Analiz)	OpenAI <code>gpt-4.1</code>	Ölçülen: soru başına ~3.200 girdi + ~700 çıktı token ≈ \$0,012

Analiz maliyeti yalnızca kullanıcı soru sorduğunda oluşur; arama ve karar listeleme hiçbir ücretli servise dokunmaz. UI her cevabın altında gerçek token kullanımını ve tahmini maliyeti gösterir (`Chat.tsx` içindeki fiyat sabitleri gpt-4.1 liste fiyatıdır: 1M girdi \$2 / 1M çıktı \$8).

İki sınıra dikkat:

- HF ücretsiz katmanının aylık kredisi var.** Limit aşırsa HF 429 döner; `src/embeddings/hf.ts` exponential backoff ile 6 kez dener, `/api/search` ise kullanıcıya anlaşılır bir mesaj gösterir. Toplu indekslemede `npm run embed` kaldığı yerden devam ettiği için koşuyu tekrarlamak yeter.
- Supabase ücretsiz projeler ~7 gün hareketsizlikte duraklatılır**, panelden elle devam ettirmek gerekir.

Bilinen sorunlar

- KVKK'nın kendi kırık linkleri.** Liste sayfalarındaki 3 kayıt (`/Icerik/5462/2019/47` , `5464/2019/52` , `5461/2019/122`) kaynak sitede anasayfaya yönleniyor; indekslenemiyor. `data/scrape-report.json` içinde `failed` altında listelenir.
- Numarasız kararlar.** Eski format bazı karar sayfalarında karar no sayfanın hiçbir yerinde geçmiyor (11 kayıt). `karar_no` bu kayıtlarda `NULL` , UI'da `no'suz` olarak gösterilir. Karar tarihi de yoksa `year` de `NULL` olur ve kayıt yıl filtresine girmez — yayınlanma tarihini karar tarihi yerine kullanmıyoruz, çünkü site 2024 tarihli bir kararı 2026'da yayınlayabiliyor.
- Birleşik kararlar.** Bazı sayfalar tek metinde birden fazla karar taşıyor (`2021/511-512-513` , `2019/81 ve 2019/165`). Bu değerler olduğu gibi saklanır.
- TLS araya giren ortamlar.** Makinede kurumsal proxy / antivirüs HTTPS taraması varsa (`NODE_EXTRA_CA_CERTS` ayarlıysa) `npm run dev` komutunu **kendi terminalinden** çalıştır. Next dev

worker'ı bu deęişkenleri yalnızca ortamdan miras alır; sanitize edilmiş bir ortamdan başlatılırsa HF çağrısı `UNABLE_TO_VERIFY_LEAF_SIGNATURE` ile patlar.

Notlar

- Scrape kibar davranır: istekler arası `SCRAPE_DELAY_MS` (400 ms) bekleme, hatalarda exponential backoff.
- `data/scrape-report.json` her koşuda üretilir: kaç karar indi, hangi meta alanları görüldü, hangi URL'ler parse edilemedi.
- Sayfa sayısı sabit deęil — scraper pagination'daki en büyük `?page=N` deęerini her koşuda yeniden okur (bugün 36).
- Bu bir resmî yayın deęildir; bağlayıcı metin için `kvkk.gov.tr` esas alınmalı.